From Local Improvement to Verified Repair: A Cross-Intervention Delivery Audit

Source-bound evidence, three-state check replay, and an exploratory prompt-only test

Artifact repository: https://github.com/ymping666/evidence-is-not-a-patch

Abstract

A check evaluates one source state; a coding agent is judged on the patch it eventually returns. We operationalize this evidence-to-artifact gap across four historical interventions: 180 episodes, 2,451 tool events, and 21 distinct tasks. Of 549 recorded checks, 213 inspect production source differing from the final artifact. Holding checks and fixtures fixed, we replay 314 qualified original/candidate/reference triples (942 executions). Among failing candidates with zero-exit checks, all three source states also exit zero in 16/16, 14/19, 16/21, and 36/36 units across panels. This is **exit-status non-discrimination**, not proof that output carries no information. Meanwhile, selective edit supervision improves validation loss by 31.5% but changes recorded repairs from 12/24 to 11/24. A separate 72-run, locally prespecified prompt-only test yields 6/24 repairs against baseline 5/24, with no stable gain. We provide source-bound measurements, three-state discrimination tests, and provenance-separated patch ablations that make local evidence and final delivery auditable.

1. Introduction

Agent optimization often targets intermediate quantities: demonstration likelihood, feedback use, generated checks, or context cost. Yet repository repair is scored on the delivered artifact. A test may evaluate an earlier source state, fail to distinguish wrong from correct code, or assess an edit that never reaches the returned patch. These are different failures of the evidence-to-artifact link.

We ask three questions: **Which source did a check inspect? Would the same check distinguish source states? Did a useful edit survive artifact handoff?** Our answer is an executable audit recipe: pair intervention outcomes with reconstructed source events, bind evidence to source hashes, replay fixed checks on three states, and examine bounded delivery counterexamples. A separate prompt-only test probes whether prompting can improve this chain; its results do not identify a causal mechanism.



Figure 1. Repair-chain audit map. Links are descriptive; correct supplied candidates may bypass editing. This is not a causal decomposition.

2. Paired interventions and outcomes

The scaffold is Henao's Budget-Fit Single Agent V5 [1], deploying Gemma-4-31B QAT. Historical complete-agent panels generally allow eight minutes, 28 counted tools, 80 turns, and 8,192 output tokens. P8 instead refines supplied candidates with eight additional tools, twelve model requests, and 480 seconds. Historical seeds were uncontrolled; P120 fixes its seed. Panels are paired within protocol, not interchangeable randomized arms.

Panel and intervention	Intermediate result	Complete-agent outcome
P5: selective edit SFT	Validation NLL 0.2670 to 0.1828 (-31.5%)	BASE 12/24; LoRA 11/24
P6.2: feedback prompt	Exploration BASE 12/16; FB 13/16	Frozen confirmation: 13/24 each
P8: candidate checking	32 independent-check outputs; zero passed the format gate or executed	R/G/D: 4/16, 4/16, 3/16
P12: native-history exclusion / smaller thinking budget	Input tokens -35.9% / -27.4%	BASE/NH4096/NH1024: 7/12, 7/12, 6/12
P120: locally prespecified prompt-only SEAL	Contract/evidence/closure instructions; no weight update	A 5/24; B 5 PASS + 1 UNKNOWN; C 6/24

Table 1. Local objectives and repairs. Counts retain scheduled attempts, including P5's twelve UNKNOWN scores. P8 measures supplied-candidate preservation. NLL validation and feedback exploration are outside the 180-episode corpus. P120's 72 runs are separate; its original and adjudicated results appear in Section 6.

Selective supervision. A rank-16 adapter learns first source-edit calls from public OpenHands trajectories: 212 units/97 tasks for training, nineteen units/ten tasks for validation, partitioned by task and repository. NLL uses the same 6,888 target tokens, including formatting, paths, and edit context. Training is BF16-dequantized; deployment is QAT. Twelve development tasks with two repeats yield two favorable and three unfavorable flips. This does not refute SFT generally.

Feedback prompting. One paragraph distinguishes invalid probes from implementation defects and requests source-level responses to behavioral mismatches. Confirmation uses twelve other development tasks twice: two wins and two losses, with repeat-level advantage reversing. Equal counts do not establish equivalence.

Candidate checking. Eight frozen candidates, six incorrect and two correct, enter refinement-only R, generated-check G, and discriminator-oriented D arms twice each. All 32 G/D outputs fail the format gate; no independent check executes. This measures a failed interface, not usable verification. All eleven passes preserve correct inputs, ten without new source edits and one after restoration. No wrong input is repaired. Scheduling deviations constrain timing comparisons.

Native history. NH4096 excludes past thought content while retaining new reasoning; NH1024 also lowers thinking to 1,024 tokens. This is not learned summarization. Input savings accompany increased output tokens and no rescued BASE failures; NH1024 loses Pyflakes. Each arm has three timeouts. These development screens establish no cross-hardware efficiency gain.

3. From interventions to a mechanical delivery audit

The event-level audit (E1) fixes P5/P6.2/P8/P12 at 48/48/48/36 episodes over 12/12/8/12 tasks, respectively: 21 distinct identities, with overlap and repeats. We report panels separately, without treating them as independent tasks or attaching independence-based confidence intervals.

We reconstruct production-source changes from diffs and tool feedback, including `write_file` and source-modifying `run_command`, not just `edit_file`. For every check, the audit binds its execution order, source hash, check/fixture identity, and outcome to a separately reconstructed final-source hash. All 180 final production-source states match archived delivery. This validates source-state reconstruction under documented assumptions, not historical loaded-module identity.

The first measurable break is **source drift**: 213/549 checks (38.8%) inspect source different from the final artifact; among 161 episodes with checks, fifteen last-check states differ. This is a temporal mismatch, not proof that the old evidence was useless.

Our strict local condition requires an executed check exiting zero **after the last actual production-source change**; within-command ordering matters. This asserts that a qualifying zero-exit event exists, not that the last check passed or the issue was fixed. An earlier zero exit cannot validate a later version.

Panel	Actual new source change	Independent PASS / FAIL / UNKNOWN	FAIL among changed episodes	FAIL after last-change zero-exit
P5	28	23 / 13 / 12	5/28	4/25
P6.2	46	26 / 22 / 0	20/46	12/36
P8	10	11 / 37 / 0	9/10	1/2
P12	31	20 / 16 / 0	11/31	10/30

Table 2. Mechanical events and original independent outcomes. The last column conditions on actual source change, a subsequent qualifying zero-exit check, and a valid final score. P8's inherited-candidate preservation bypass is essential: no new edit need be appropriate for an already-correct candidate.

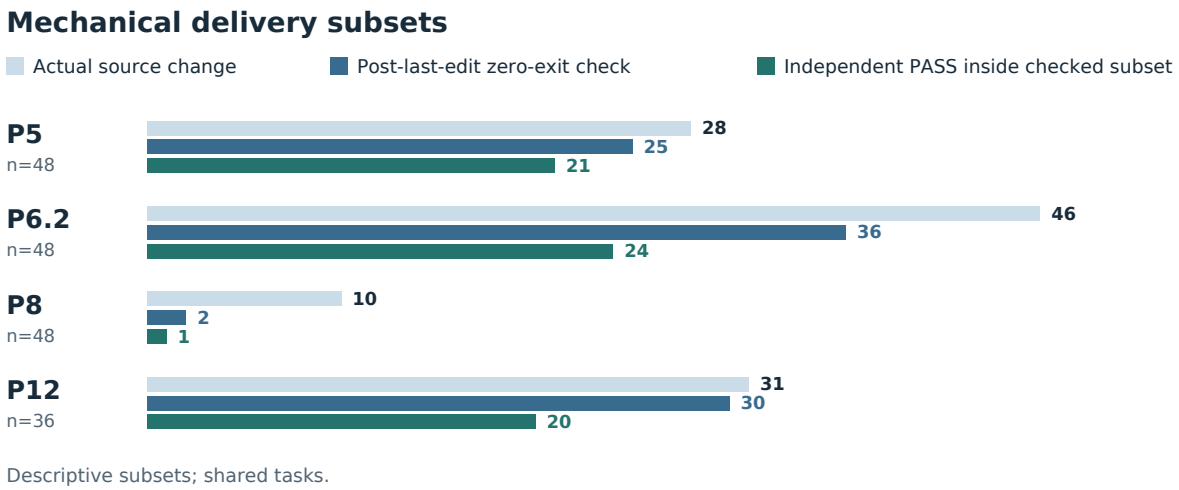


Figure 2. Nested descriptive subsets - source change, a qualifying later zero exit, and independent PASS - not necessary steps for every success. P8's one PASS inside the change branch restores an already-correct input. The broader any-after-change failure counts, 4/25, 14/38, 4/5, and 10/30, remain appendix diagnostics; especially P8's 4/5 must not replace the strict 1/2.

4. Three-state discrimination and temporal alignment

The three-state audit (E4) asks whether an unchanged agent-written check distinguishes **original buggy code, the final agent candidate, and reference repair**. Command, check body, auxiliary fixtures, and qualified environment remain fixed; only the source changes. Identities include hashes and the complete candidate patch. Source hashes before fixtures, after fixtures, and after checking guard against accidental candidate mutation. Invalid application, missing inputs, and environment failures are UNKNOWN.

Eligibility is frozen before replay outcomes: 318 candidate-conditioned units yield 314 eligible triples, covering 408 historical events/130 episodes; four have incomplete or inconsistent fixtures. All 942 canonical executions qualify. Selection favors safely recoverable checks and is not representative of all checks.

Panel	Candidate exit=0 / failed-candidate units	All-three-zero / candidate-zero units	Failed episodes covered	Post-last-change candidate-zero / historical events
P5	16/20 (80.0%)	16/16 (100%)	8/13	9/9
P6.2	19/28 (67.9%)	14/19 (73.7%)	11/22	12/16
P8	21/29 (72.4%)	16/21 (76.2%)	23/37	0/1
P12	36/45 (80.0%)	36/36 (100%)	16/16	20/20

Table 3. Candidate-conditioned exit-status discrimination. Units deduplicate check/fixture/candidate/environment identities within panel. The all-three-zero column conditions further on candidate exit=0: 36/36 in P12 means all thirty-six zero-exit checks also return zero on original and reference, not thirty-six additional episodes. Reference nonzero outcomes do not remove candidates from the first denominator. Repeated tasks and checks remain dependent.

The matrix reveals **candidate-conditioned status blindness**: many failing-candidate checks also exit zero on both original and reference source. Printed errors or caught exceptions may still expose differences. A zero exit can also correctly validate one sub-behavior while another requirement fails. Conversely, four P12 units exit zero on original/reference but nonzero on the candidate; omitting the candidate would miss this distinction.

Table 3's last column separately restricts to the check's **historical position after the last source change**, with event rather than unit weighting. These are new-environment reexecutions, not original model-visible feedback. Small denominators, especially P8, limit comparison. P8 often preserves a supplied candidate without new source editing. Neither subset estimates historical agent misjudgment rates.

Event-weighted and fixture-excluded sensitivity counts are released in the statistical supplement; they do not imply complete historical-workspace recovery. New-environment independent labels change only P5 to 20/24 units; original UNKNOWN scores remain.

These measurements separate source drift from exit-status non-discrimination. Neither alone establishes that a check is behaviorally irrelevant, and the qualified replay subset must not be extrapolated to all historical checks.

5. Bounded source-state and commitment cases

The source-state case audit (E2) scores twenty qualified tasks: 166 new final scores agree in all 154 cases with comparable valid original labels, while fourteen Selfies episodes remain unqualified; 149/153 intermediate attempts score validly. P5 Billiard creates and later loses a passing intermediate patch. P8 OpenAPI instead reverts an initially supplied correct patch. Three captured self-checks from these cases exit zero on both passing and failing states, although printed output may differ. These are **existence cases**, not failure-rate estimates.

The budget-denied-edit replay (E3) distinguishes **workspace repair from submission**. P6.2 Pyflakes rejects an edit because `old_string` is absent. In P12 Pyflakes, a denied edit would yield a 25/25 test-passing workspace patch, but it occurs at step 32, 34.21 seconds *after* `submit_patch` at step 31. The submitted 238-byte patch contains only a reproduction script. The official runner returns that existing snapshot; no resubmission or end-to-end counterfactual delivery was executed. Thus extra edit allowance is not a demonstrated rescue.

6. Prospective exploratory SEAL intervention

SEAL (Specification-grounded Evidence and Artifact Locking) changes only `system.md`: A is exact BF_V5; B adds Stop & Seal budget/closure guidance; C additionally requests an issue-grounded contract and final-source evidence. Weights, tools, and scaffold are fixed. Tasks, prompts, seed 20261018, contrasts, and UNKNOWN handling were hash-locked locally *before* main runs, not publicly preregistered. Six exposed engineering smokes were excluded; their scores did not select prompts.

Twenty-four repositories receive one A/B/C run each: **72 contemporary generations**, no reused historical outputs. Arms share the GPU/profile; six arm orders repeat four times. Limits are 480 seconds, 28 charged tools, 80 turns, 8,192 output and 4,096 thinking tokens, with temperature/top-p 0.2/0.95. These are 22 reused DEV tasks and two TRAIN-origin candidates with no exposure found in a bounded scan, **not** 24 fresh heldout tasks.

Original outcomes are A: 5 PASS/19 FAIL; B: 5 PASS/18 FAIL/1 UNKNOWN; C: 6 PASS/18 FAIL. C versus A has one win, no losses (unadjusted exact discordant p=1.0); C versus B has prespecified net bounds of 0-1 repair. All arms fail both project-unexposed candidates. The sole flip, Abjad, is reused DEV; there is no stable gain.

B's Marshmallow collection error remains UNKNOWN in the primary record. Later CPU controls attribute it to a candidate-added double-`cls` call; removing that exception is not a repair. Separate assistant adjudication labels B FAIL, yielding A/B/C totals of 5/5/6, but still only **71 valid original scores**, not 72.

In Abjad, C executes a repository example for the second reported behavior, interprets its output, then changes a constructor default and replaces mutation-during-iteration with filtering. A/B also discuss that behavior, but A implements only filtering and B retains debugging. A/B fail two default-value assertions; C passes eight tests. Post-hoc CPU ablations use the unchanged frozen scorer: default-only passes 8/8; filtering-only passes 6/8 and fails the same two constructor-default assertions. The default change alone reproduces C's test success. This isolates code contributions under existing tests, not prompt causality.

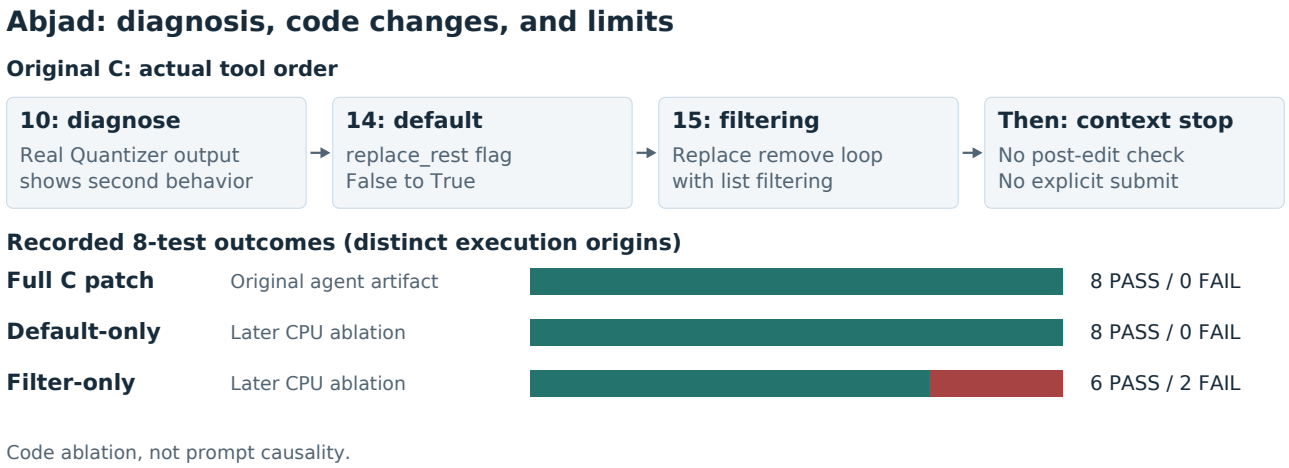


Figure 3. Abjad evidence and patch ablation. C's original agent artifact and the two later CPU ablations have distinct provenance. Only the default-value change is needed for the existing eight-test PASS; this does not identify prompt causality or broader correctness. C never performed post-edit verification or explicitly submitted.

No failing check occurs between C's two edits. C performs no post-edit check; all three arms exceed context and never explicitly submit. Its PASS is the returned workspace patch, not demonstrated final-verification or commitment success. This is an exploratory diagnosis-to-action case, with unfulfilled SEAL mechanisms.

7. Related work and scope of novelty

SWE-bench supplies execution-based repository-repair evaluation [2]. SWE-Gym trains agents and trajectory-based verifiers [3]; SWE-Prime selects trajectories and loss-bearing segments while preserving context [4]. They establish effective learning approaches, whereas our selective-edit result separates improved conditional target fit from complete-agent outcomes; it neither reproduces nor refutes their methods.

SWE-Replay reuses trajectory prefixes and branches to improve test-time scaling [5]. Our replay independently scores archived source states; it does not sample policy continuations or estimate recovery probability. ExecCritic separates and trains test construction and repair, qualifying and freezing generated tests [6]. Our fixed-check experiment audits discrimination retrospectively; P8's failed format gate provides no comparison with usable independent-test feedback.

Trajectory research already studies successful versus failed strategies [7], temporal onset and recovery of failures [8], and divergence between submission and test-verified resolution, including edits to already-correct code [9]. Trial by Trace separately evaluates harness reliability and task success [10]. Our contribution links intervention outcomes to verifiable delivery, candidate-conditioned temporal check matrices, and a weak prospective prompt-only test. We claim neither the first failure taxonomy nor causal attribution to individual harness or prompt components.

8. Limitations, artifacts, and conclusion

The historical panels share development tasks and differ in protocol; they do not establish cross-task generalization or the ineffectiveness of fine-tuning and verification. Replay relies on reconstructed source and qualified environments, with eligibility bias and possible probe-internal randomness. No cross-hardware timing or independence-based significance is claimed.

Semantic inspection was targeted: 148 assistant-inspected units, including 61 checks; four core historical cases received additional claim-level review. This is not exhaustive annotation, and there were **zero human confirmations**. Detailed labels and supplementary counts are documented in the public artifact.

The public artifact provides anonymized statistics, source hashes, Abjad patches, and CPU verification scripts. It excludes private tests, the task scorer, full snapshots, and raw trajectories. Statistical recomputation is not new agent execution; reproducing private test outcomes still requires unavailable scoring assets.

The result is a **source-bound audit protocol** that exposes separate breaks in source continuity, check discrimination, and artifact delivery. It turns an intuitive last-mile concern into inspectable measurements and concrete counterexamples. SEAL remains exploratory; future controlled studies must test whether repairing these breaks improves independent issue resolution.

References

1. Henao, W. E. (2026). *Gemma 4 Agent: Budget-Fit Single Agent* (Version 5). Kaggle Notebook.

2. Jimenez, C. E., et al. (2024). *SWE-bench: Can Language Models Resolve Real-World GitHub Issues?*. ICLR.

3. Pan, J., et al. (2025). *Training Software Engineering Agents and Verifiers with SWE-Gym*. ICML, PMLR 267:47717-47737.

4. Zheng, D., et al. (2026). *SWE-Prime: Fewer Trajectories, Better Performance*. arXiv:2608.27449.

5. Ding, Y. and Zhang, L. (2026). *SWE-Replay: Efficient Test-Time Scaling for Software Engineering Agents*. arXiv:2601.22129.

6. Tao, L., et al. (2026). *ExecCritic: Learn to Test, Test to Improve for Coding Agents*. arXiv:2609.09133.

7. Majgaonkar, O., et al. (2025). *Understanding Code Agent Behaviour: An Empirical Study of Success and Failure Trajectories*. arXiv:2511.00197.

8. Zhao, X., et al. (2026). *Failure as a Process: An Anatomy of CLI Coding Agent Trajectories*. arXiv:2607.09510.

9. Mehta, A. (2026). *Confident and Wrong: Silent Semantic Failures in Coding Agents*. arXiv:2603.25764.

10. Cao, Z., et al. (2026). *Trial by Trace: Benchmarking AI Agent Harnesses for Reliability and Failure Attribution*. arXiv:2610.32285.